

PRACTICAL 7

Coin Change Problem using Dynamic Programming

Objective: Find the minimum number of coins required to make a given amount using Dynamic Programming.

Python Program

```
# Coin Change Problem using Dynamic Programming

def coin_change(coins, amount):

    # dp[i] = minimum coins required to make amount i
    dp = [float('inf')] * (amount + 1)

    # Base case: 0 coins are needed to make amount 0
    dp[0] = 0

    # Build the DP table
    for i in range(1, amount + 1):
        for coin in coins:
            if coin <= i:
                dp[i] = min(dp[i], dp[i - coin] + 1)

    # If amount cannot be formed
    if dp[amount] == float('inf'):
        return -1

    return dp[amount]

# Taking input from user
n = int(input("Enter number of coins: "))

coins = []
print("Enter the coin values:")

for i in range(n):
    coin = int(input(f"Coin {i + 1}: "))
    coins.append(coin)

amount = int(input("Enter the amount: "))

# Calculate answer
answer = coin_change(coins, amount)

print("Minimum number of coins:", answer)

print("Time Complexity : O(n * amount)")
print("Space Complexity : O(amount)")
```

Sample Output

```
Enter number of coins: 3
Enter the coin values:
Coin 1: 2
Coin 2: 1
Coin 3: 5
Enter the amount: 8
Minimum number of coins: 3
Time Complexity : O(n * amount)
```

Space Complexity : $O(\text{amount})$

Result: For coins {2, 1, 5} and amount 8, the minimum number of coins is 3.